



Enterprise Web Server Deployment



Installing & Configuring Nginx

- RHEL
- CentOS
- Ubuntu



AGENDA

- Introduction & Prerequisites
- Setup a Virtual Machine (Rocky / CentOS / Ubuntu)
- Step 1: Installing EPEL Repository & Nginx
- Step 2: Managing the Nginx Service
- Step 3: Configuring the Firewall
- Step 4: Setting Up Virtual Hosts
- Step 5: Creating the Server Block Configuration
- Verification & Next Steps



PREREQUISITES & INITIAL SETUP

- Before we begin, ensure the system is ready:
- Operating System: RHEL 8/9 or CentOS 8/9
- User Access: Root or Sudo privileges
- Network: Internet access for package downloads



Initial Package Updates:

- `dnf install -y epel-release`
- `dnf update -y`

Install NGINX

- `dnf install nginx -y`
- `nginx -v` (check the version)
 - **Expected Output: (nginx version: nginx/1.20.1 (or later))**

Managing The NGINX SERVICE

- After installation, we need to start the service and ensure it runs automatically

Command	Purpose
Systemctl status nginx	Check if Nginx is currently active
Systemctl enable nginx	Configure Nginx to start at system boot
Systemctl start nginx	Start the Nginx service immediately
Systemctl status nginx	Verify the service is running successfully

FIREWALL-CMD CONFIGURATION

- We must allow web traffic through the firewall to make the server accessible
- Commands are:
- `firewall-cmd - -permanent - -add-service=http`
- `firewall-cmd - -permanent - -add-service=https`
- `firewall-cmd - -reload` (very important)
- `Firewall-cmd - -list-all` (to check the list)

FIREWALL VERIFICATION OUTPUT

```
[root@localhost html]# firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: cockpit dhcpv6-client http https ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

Confirmation: The services line now includes http and https

DIRECTORY STRUCTURE & PERMISSIONS

- We create a dedicated directory structure for our website.
- For our example domain yogurt.com:
- `mkdir -p /var/www/yougurt.com/html`
 - **mkdir -p** : Creates the parent directories if they don't exist
- `chown -R $USER:$USER /var/www/ yougurt.com/html`
 - **chown** : Assigns ownership to the current user
- `chmod -R 755 /var/www/ yougurt.com/`
 - **chmod** : Sets read/execute permissions for the group and others

SELINUX CONTEXT CONFIGURATION

- For security, SELinux requires proper file contexts.
- Commands:
- `semanage fcontext -a -t httpd_sys_content_t "/var/www/business.com/html(/.*)"?"`
- `restorecon -R /var/www/business.com/html/`
- **Note: If you get a semanage command not found error, install it using:**
 - `dnf install policycoreutils-python-utils`

CREATING THE SERVER BLOCK

- The main configuration is done via a server block file in `/etc/nginx/conf.d/`
- Naming convention: Use the domain name for clarity (e.g., `yogurt.conf`)
- Create the configuration file: `(domain.com.conf)`
- `nano /etc/nginx/conf.d/yogurt.com.conf`
- Config file on a next slide

SERVER BLOCK CONFIGURATION

- Here is the standard configuration for our virtual host

```
[root@localhost conf.d]# cat yogurt.com.conf
server {
    listen 80;
    listen [::]:80;
    server_name yogurt.com www.yogurt.com;

    root /var/www/yogurt.com/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    # Optional: Custom error pages
    error_page 404 /404.html;
    location = /404.html {
        internal;
    }

    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        internal;
    }
}
```

- listen : Accepts connections on port 80 (IPv4 & IPv6)
- server_name : Defines which domain names this block responds to
- root : Points to the directory where your site files are stored

Setup Different Domain

- Applying the exact same steps for a different domain
- Assuming our domain is xyz.com:
- `mkdir -p /var/www/xyz.com/html`
- `chown -R $USER:$USER /var/www/xyz.com/html`
- `chmod -R 755 /var/www/xyz.com`
- `nano /etc/nginx/conf.d/xyz.com.conf`
- `semanage fcontext -a -t httpd_sys_content_t "/var/www/xyz.com/html(/.*)?"`
- `restorecon -R /var/www/xyz.com/html/`
- Conclusion: The process is identical for any domain; simply replace the domain name

TESTING & RELOADING

- `nginx -t`
- Look for: syntax is ok and test is successful
- Reload Nginx:
- `systemctl reload nginx`
 - (Reload applies changes without dropping existing connections)



SUMMARY & KEY TAKEAWAYS

- Foundation: EPEL repository is critical for installing Nginx on RHEL-based systems
- Service Management: Always enable and start services to ensure persistence
- Security: Firewall rules and SELinux contexts are essential for a secure deployment
- Virtual Hosts: Each domain gets its own directory and configuration file
- Testing: Always test your configuration (`nginx -t`) before reloading



Thank You!